

IoT Asset Tracking and Sensor Monitoring System

Software Design Document

Mohamed Tawfiq Cassim

Table of Contents

Introduction	3
Statement.....	3
Functional Requirements	3
Tech Stack	3
System Design	4
Application Architecture	5
Features	5
Challenges & Solutions	6
Future Improvements	6
Conclusion	6

Introduction

The project presents the design and development of an IoT Asset Tracking and Monitoring System, specifically for a company that develops IoT tracking hardware and software. This system allows functionality such as managing firmware versions, organising devices into hierarchical groups, and displaying all registered IoT devices for user simplicity. It was developed using Visual Studio ASP.NET Core MVC, with MSSQL Server as the relational database and Entity Framework Core using the database-first approach.

Statement

The company manufactures multiple IoT devices with varying capabilities, while each device runs a specific firmware version and belongs to an organisational group. The system uses a centralised platform capable of managing firmware, hierarchical groups and associated devices while maintaining data integrity through a relational database.

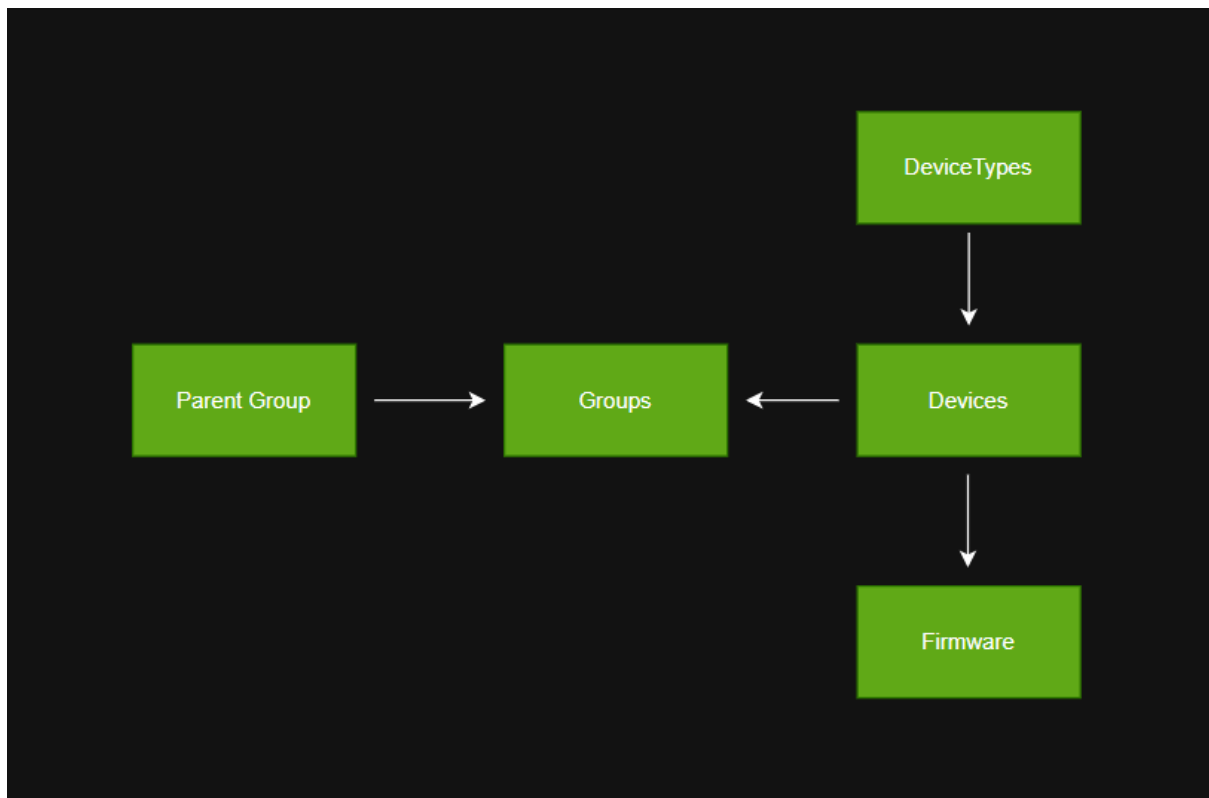
Functional Requirements

Requirement	Implemented
Hierarchical Groups	Yes
Firmware Versions	Yes
Device link to firmware	Yes
Device link to group	Yes
Device Types	Yes
Firmware CRUD	Yes
Group CRUD	Yes
Device Viewing	Yes

Tech Stack

Technology	Reason
C#	Strong supported language for .Net
ASP.Net Core MVC	Presentation, logic, and data are all separated
MSSQL Server	Relational db with strong support
Entity Framework Core	Simplifies db access
Bootstrap 5	Creates a consistent and supportive UI

System Design



The system was designed using a relational database, each entity within the database represents a key component of the IoT asset management system, with relationships created through primary and foreign keys.

Central Entity

- **Device:** The central table where all device-related information is stored. Each device connects to exactly one Firmware, one Group, and one Device Type to keep data centralized and duplication-free.

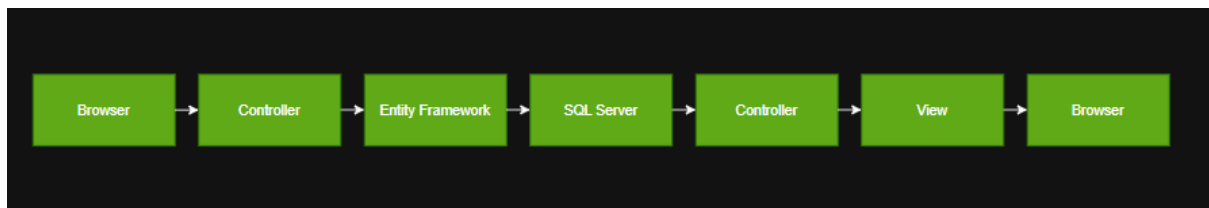
Supporting Tables

- **Firmware:** Stores software versions. Multiple devices can use the same firmware version (one-to-many relationship).
- **Group:** Organizes devices hierarchically (like a tree structure) using a self-referencing relationship. This allows you to map out real-world setups like Regions → Warehouses → Fleets.
- **DeviceType:** Manages the different categories of IoT devices. Keeping this separate makes it easy to add new device types down the road without messing up current records.

Design Standards

- **Third Normal Form (3NF):** The database is normalized so each table has a single responsibility, which eliminates redundant data and protects data integrity.
- **Foreign Keys:** Constraints are in place to ensure all relationships remain valid, preventing orphaned data and keeping the entire system consistent.

Application Architecture



Core Architecture

- **Model-View-Controller (MVC):** Built on ASP.NET Core, this pattern splits the app into three distinct layers. This separation makes the system easier to develop, test, maintain, and expand.

The Three Components

- **Controller:** Acts as the middleman. It intercepts user requests (like clicking a link), processes the logic, talks to the data layer via Entity Framework, and decides which View to show next.
- **Model (via Entity Framework Core):** Handles the data. It uses a Database First approach, meaning the code models were automatically generated directly from the SQL Server database. This keeps the application perfectly synced with the database schema and avoids manual coding errors.
- **View:** Handles what the user sees. It uses Razor pages to display the data provided by the controller, keeping user interface (UI) presentation completely separate from business logic. Bootstrap is included to ensure the layout is clean and responsive on all devices.

Key Benefits

- **Separation of Concerns:** You can modify the UI, the business logic, or the database independently without breaking the other pieces.
- **Best Practices:** This approach cuts down on messy code, boosts code reusability, and makes long-term maintenance much easier.

Features

- **Dashboard:** Functions as the central landing page and navigation hub. It provides an immediate overview of live system statistics (total devices, firmware, groups, and device types) and includes quick-action buttons to reduce clicks for common tasks.
- **Firmware Management:** Provides full CRUD (Create, Read, Update, Delete) capabilities for firmware versions, tracking the version number, release date, and change description. Storing this in a dedicated table reduces data duplication since multiple devices share the same firmware.
- **Group Management:** Allows devices to be organized into a tree-like hierarchy (e.g., Region → Warehouse) using a self-referencing relationship. It includes full CRUD functionality to make system scalability, reporting, and administration easier.

- Device List: Provides a read-only view of all registered devices, displaying key details like serial numbers, types, firmware, and groups. It uses Entity Framework eager loading to pull data across tables, proving that the database's foreign key relationships work correctly.

Challenges & Solutions

Challenge	Solution
MVC Location (Views) issue	I tried calling from a folder while it was empty, sorted that out by moving the views to the correct folders.
NullReferenceException thrown	The models weren't passing correctly through the controller; I fixed that by rendering the correct view.
Dependency injection conflict	Solution generated views with multiple constructors, fixed that by removing all duplicated constructors.
Understanding the relationships	Struggled at first to find the correct relationships between the entities, I managed to find the correct relationships by breaking them down into more generic details.

Future Improvements

- Device CRUD Management – Currently it is read-only, an improvement to make in the future would be allow administrators to add new devices, modify existing devices, and delete unused/outdated devices.
- User Authentication – Would secure the system better and better insure data safety.
- Firmware Update History – Produce a record for all updates done to the firmware of each device, to allow for better tracking and review.
- Search/Sorting Methods – Search functionality to allow users to filter through devices, groups, device types, and firmware versions, to quick location across big data/large quantity of devices.

Conclusion

The project was a success and met all the requirements by implementing a relational SQL Server database and an ASP.NET Core MVC web application capable of managing firmware and hierarchical groups, and displaying all IoT device information. This solution shows database design, effective use of Entity Framework Core, and a maintainable MVC architecture, while keeping the final application functional, scalable, and a solid foundation for any future enhancements.